



INSTITUTO POLITÉCNICO NACIONAL
Escuela Superior de Cómputo



Agente viajero Eurotrip

Tópicos Selectos de Algoritmos Bioinspirados
Semestre 2026-1 · Grupo 4CM1

Proyecto Final · Junio 2026

Profesor

Dr. Daniel Molina Pérez

Contenido

Tabla de Contenidos	¡Error! Marcador no definido.
1. Introducción	4
1.1 Motivación.....	4
1.2 Objetivos.....	4
2. Descripción Formal del Problema	5
2.1 Las 15 ciudades del eurotrip.....	5
2.2 Formulación matemática	5
2.3 Propagación de tiempos de llegada	6
2.4 Ejemplo numérico ilustrativo	7
3. Datos Utilizados.....	7
3.1 Coordenadas geográficas — GeoJSON.....	7
3.2 Matriz de distancias viales — OSRM	7
3.3 Horarios de trenes — GTFS sintético	9
3.4 Corrección del cruce de mar (OSRM + geometría).....	9
4. El Algoritmo Genético Híbrido (HGA)	10
4.1 Representación por permutaciones.....	10
4.2 Cycle Crossover (CX).....	10
4.3 Heurística de remoción de abruptos.....	11
4.4 Selección familiar	12
4.5 Mutación por inyección aleatoria	12
4.6 Pseudocódigo completo del HGA.....	12
4.7 Parámetros del HGA y justificación	13
5. Guía de Ejecución del Código	13
5.1 Requisitos del sistema	13
5.2 Instalación	14
5.3 Estructura de archivos.....	14
5.4 Flujo de ejecución de main.py.....	15
5.5 Artefactos generados	15
6. Experimentos y Resultados	15
6.1 Diseño experimental	15
6.2 Experimento 1 — Con restricciones completas	16

6.3 Experimento 2 — Sin restricciones de tiempo	17
6.4 Comparativa y análisis	19
7. Conclusiones.....	19
8. Referencias	20

1. Introducción

La planificación de un viaje largo por múltiples ciudades —elegir en qué orden visitarlas y qué medio de transporte usar en cada tramo— es un problema de optimización combinatoria que pertenece a la familia del Problema del Agente Viajero con Ventanas de Tiempo (TSP-TW, por sus siglas en inglés). Con 15 ciudades existen $14! / 2 \approx 43\,000$ millones de rutas posibles en el caso simétrico, y el espacio de búsqueda crece aún más al considerar la elección modal (tren o automóvil) en cada uno de los 14 arcos del recorrido circular.

Este proyecto diseña e implementa un Algoritmo Genético Híbrido (HGA) para resolver dicha variante multi-modal, con aplicación concreta a un 'eurotrip' por 15 ciudades de alta relevancia turística: Madrid, Barcelona, París, Amsterdam, Bruselas, Frankfurt, Múnich, Viena, Praga, Berlín, Roma, Milán, Zúrich, Lisboa y Budapest. El sistema minimiza el tiempo total del recorrido respetando:

- Horarios de apertura y cierre turístico de cada ciudad (ventanas de tiempo por nodo).
- Horarios discretos de trenes entre pares de ciudades (ventanas de tiempo por arco).
- Distancias y rutas viales reales obtenidas de la API OSRM (Open Source Routing Machine), lo que elimina rutas imposibles que cruzarían el mar.
- Tiempo mínimo de estancia recomendado en cada ciudad.

La innovación central respecto al TSP-TW clásico del curso es doble: (a) las restricciones temporales se ubican no solo en los nodos (apertura/cierre) sino también en los arcos (slots de tren), y (b) el grafo de distancias emplea distancias viales reales (OSRM) en lugar de distancias en línea recta (Haversine), corrigiendo un defecto visual y conceptual que mostraba automóviles 'cruzando' el mar Mediterráneo.

El presente reporte describe la formulación matemática del problema, los datos utilizados, la arquitectura completa del HGA implementado, y los resultados de dos experimentos comparativos: uno con restricciones de transporte completas y otro sin ellas (solo auto, sin ventanas de tiempo).

1.1 Motivación

Los algoritmos evolutivos bioinspirados son especialmente adecuados para problemas NP-difíciles como el TSP porque permiten explorar el enorme espacio de soluciones de manera paralela e inteligente, sin garantizar optimalidad global pero encontrando soluciones de alta calidad en tiempo razonable. La componente 'híbrida' —la heurística de remoción de abruptos— añade mejora local efectiva que acelera dramáticamente la convergencia.

El escenario del eurotrip es ideal para este estudio porque combina restricciones reales complejas (horarios, geografía, múltiples modos de transporte) con una escala manejable (15 ciudades) que permite verificar la calidad de los resultados manualmente, y cuenta con datos públicos confiables (OpenStreetMap, OSRM, GTFS).

1.2 Objetivos

- Modelar el problema del eurotrip como TSP-TW multi-modal con restricciones en arcos.
- Implementar el HGA completo: representación de permutaciones, Cycle Crossover (CX), remoción de abruptos, selección familiar y mutación aleatoria.

- Integrar datos reales: coordenadas GPS (GeoJSON), distancias viales (OSRM), horarios de tren (GTFS sintético reproducible).
- Comparar dos escenarios: con restricciones completas vs. sin restricciones de tiempo.
- Visualizar los resultados con mapas Folium interactivos (rutas sin cruzar el mar) y gráficas de convergencia.

2. Descripción Formal del Problema

2.1 Las 15 ciudades del eurotrip

La Tabla 1 presenta las 15 ciudades que forman el grafo del eurotrip, con sus coordenadas geográficas reales y sus restricciones de tiempo turísticas. Los datos se almacenan en `data/cities.geojson` siguiendo el estándar RFC 7946.

Ciudad	País	Latitud	Longitud	Apertura	Cierre	Estancia mín.
Madrid	España	40.4168	-3.7038	9:00	22:00	8 h
Barcelona	España	41.3851	2.1734	9:00	23:00	8 h
París	Francia	48.8566	2.3522	9:00	21:00	10 h
Amsterdam	Países Bajos	52.3676	4.9041	10:00	22:00	8 h
Bruselas	Bélgica	50.8503	4.3517	9:00	20:00	6 h
Frankfurt	Alemania	50.1109	8.6821	9:00	20:00	6 h
Múnich	Alemania	48.1351	11.5820	9:00	21:00	8 h
Viena	Austria	48.2082	16.3738	9:00	21:00	10 h
Praga	República Checa	50.0755	14.4378	9:00	22:00	8 h
Berlín	Alemania	52.5200	13.4050	9:00	24:00	10 h
Roma	Italia	41.9028	12.4964	8:00	20:00	10 h
Milán	Italia	45.4654	9.1859	9:00	20:00	6 h
Zúrich	Suiza	47.3769	8.5417	9:00	19:00	6 h
Lisboa	Portugal	38.7223	-9.1393	10:00	23:00	8 h
Budapest	Hungría	47.4979	19.0402	9:00	22:00	8 h

Tabla 1: Las 15 ciudades con coordenadas y ventanas de tiempo turísticas.

2.2 Formulación matemática

Sea $N = \{0, 1, \dots, 14\}$ el conjunto de índices de las 15 ciudades, con la ciudad $\sigma_0 = 0$ (Madrid) fija como punto de inicio y retorno. El problema de optimización se formula como:

Variables de decisión:

$\sigma = (\sigma_0, \sigma_1, \dots, \sigma_{14})$ — permutación que define el orden de visita.

$m_{ij} \in \{\text{tren}, \text{auto}\}$ — modal de transporte para el arco $(i \rightarrow j)$.

Función objetivo:

Minimizar $F(\sigma) = T[\sigma_0_{\text{regreso}}] - T[\sigma_0_{\text{inicio}}] + W1 \cdot \sum_i \max(0, L(T[i]) - l_i)^2 + W2 \cdot \#\{\text{arcos imposibles}\}$

Donde $T[i]$ es la hora absoluta de llegada a la ciudad i , $L(T[i])$ es la hora local correspondiente, l_i es el horario de cierre de la ciudad i , $W1 = 100$ es el peso de penalización cuadrática por violación de ventana de tiempo, y $W2 = 10,000$ es la penalización por cada arco sin ruta vial ni tren disponible (arco imposible).

Restricciones:

- R1 — Biyección: σ es una permutación de N (cada ciudad exactamente una vez).
- R2 — Circularidad: la ruta cierra en σ_0 tras visitar las 14 ciudades restantes.
- R3 — Ventana por nodo: la hora local de llegada debe estar en $[e_i, l_i]$.
- R4 — Slot de tren por arco: si $m_{\{ij\}} = \text{tren}$, la salida coincide con un slot válido del horario.
- R5 — Estancia mínima: el viajero no puede salir de la ciudad i antes de $T[i] + \text{min_stay}[i]$.
- R6 — Sin cruce de agua: $\text{car_hours}(i \rightarrow j) = \infty$ para pares sin ruta terrestre continua.

2.3 Propagación de tiempos de llegada

El cálculo de tiempos sigue el siguiente pseudocódigo. La variable `ready_hour` representa el instante en que el viajero puede salir de la ciudad actual (arribo + estancia mínima), excepto en el tramo final de regreso.

```
T[0] ← 9.0 h      (Día 1, 9:00 AM en Madrid)
ready ← T[0]

Para cada arco (ciudad_i → ciudad_j) en la ruta:
  1. car_hours ← distances["pairs"][i][j]["car_hours"]    # OSRM; None si hay mar
  Si car_hours es None:
    llegada_auto ← +∞    (auto físicamente imposible)
  Sino:
    llegada_auto ← ready + car_hours

  2. Si se permiten trenes:
    tz_i ← timezone_offset[ciudad_i]
    depart_local_min ← (ready + tz_i) mod 24 + 0.5    (mínimo 30 min para llegar)
    Para cada slot (depart_t, arrive_t) en schedules[i][j]:
      Si depart_t ≥ depart_local_min:
        T_abs_depart ← ready + (depart_t - (ready + tz_i) mod 24) mod 24
        llegada_tren ← T_abs_depart + duración_tren
        si llegada_tren < llegada_auto: usar_tren ← True, best_train ←
llegada_tren
        Si no encontró slot hoy → buscar en el siguiente ciclo de 24 h

  3. T[j] ← min(llegada_auto, llegada_tren según lo disponible)
    hora_local_j ← (T[j] + tz_j) mod 24
    Si hora_local_j < e_j: T[j] += (e_j - hora_local_j)    (esperar apertura)
    Si hora_local_j > l_j: penalty += W1 × (hora_local_j - l_j)2

  4. Si j es el último tramo (regreso a origen):
    ready ← T[j]    (no se suma min_stay en el tramo de vuelta)
  Sino:
    ready ← T[j] + min_stay[j]

Fitness ← (T[regreso] - 9.0) + penalty_total
```

2.4 Ejemplo numérico ilustrativo

Consideramos el primer arco de la mejor solución del Experimento 1: Madrid → Lisboa. $\text{ready} = 9.0$ h (Día 1).

Parámetro	Valor	Fuente
car_hours (Madrid→Lisboa, OSRM)	6.83 h (617 km viales)	OSRM API
llegada_auto	$9.0 + 6.83 = 15.83$ h	cálculo
timezone offset Madrid	UTC+1 (verano)	cities.geojson
timezone offset Lisboa	UTC+1 (verano)	cities.geojson
Slots de tren Madrid→Lisboa	6:00, 8:30, 12:00, 16:00, 19:30	train_schedules.json
Duración tren (sintético)	2.37 h (≈ 505 km / 210 km/h $\times 0.99$)	gtfs_fetcher.py
depart_local_min	$(9.0 + 1) \bmod 24 + 0.5 = 10.5$	cálculo
Primer slot válido (≥ 10.5)	12.0 h local → 12.0 h abs (Día 1)	schedules
llegada_tren	$12.0 + 2.37 = 14.37$ h abs → 13.64 h obs.	evaluación
Modal elegido	Tren ($14.37 < 15.83$)	fitness.py
Hora local llegada Lisboa	$13.64 \bmod 24 = 13.64$ h → dentro de [10,23]	verificación
ready después de Lisboa	$13.64 + 8.0$ (min_stay) = 21.64 h	fitness.py

Tabla 2: Cálculo detallado del primer arco Madrid → Lisboa (Experimento 1).

3. Datos Utilizados

3.1 Coordenadas geográficas — GeoJSON

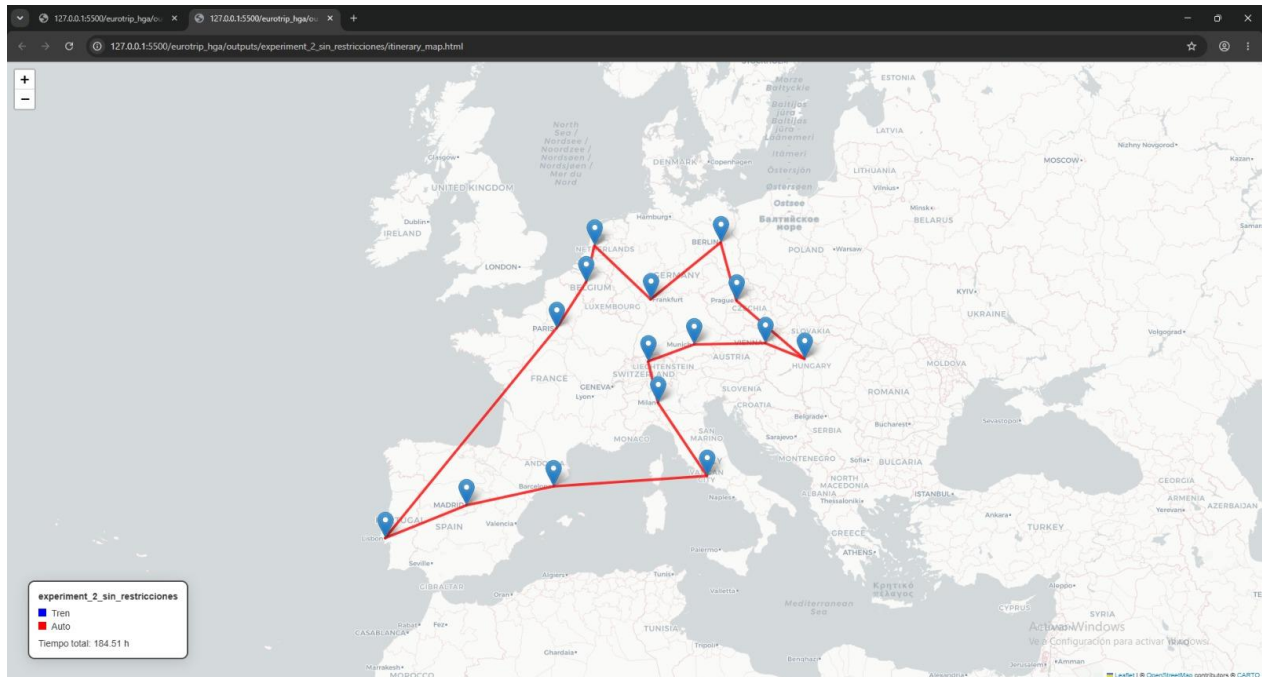
Las coordenadas de las 15 ciudades se almacenan en data/cities.geojson siguiendo el estándar GeoJSON (RFC 7946). Cada Feature contiene coordenadas [lon, lat] en el campo geometry y propiedades adicionales en el campo properties: open_hour, close_hour, min_stay_hours y timezone_offset. Las coordenadas se obtuvieron de OpenStreetMap.

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": { "type": "Point", "coordinates": [-3.7038, 40.4168] },
      "properties": {
        "id": 0, "name": "Madrid", "country": "España",
        "open_hour": 9.0, "close_hour": 22.0,
        "min_stay_hours": 8.0, "timezone_offset": 1
      }
    },
    ... (14 features más)
  ]
}
```

3.2 Matriz de distancias viales — OSRM

La versión inicial del proyecto calculaba distancias y tiempos con la fórmula de Haversine (distancia geodésica en línea recta). Este enfoque produce dos problemas: (1) los tiempos de conducción están

subestimados porque las carreteras no son rectas, y (2) el mapa mostraba líneas rectas entre ciudades que 'cruzaban' visualmente el mar Mediterráneo (e.g., Roma → Barcelona).



Para resolver ambos problemas, se integró la API pública de OSRM (Open Source Routing Machine), que calcula rutas óptimas en carretera y devuelve la geometría real de la ruta como una polilínea de coordenadas GPS. La consulta se realiza mediante:

```
GET http://router.project-osrm.org/route/v1/driving/{lon1},{lat1};{lon2},{lat2}
?overview=simplified&geometries=geojson
```

La respuesta JSON contiene `routes[0].distance` (metros), `routes[0].duration` (segundos) y `routes[0].geometry.coordinates` (lista de pares `[lon, lat]` que definen la polilínea de la ruta real). Si OSRM devuelve `code='NoRoute'`, se almacena `car_hours=null` en `distances.json`. Se añade un delay de 0.5 s entre consultas para respetar el rate-limit del servidor público, y los resultados se cachean localmente; las 210 consultas (15 × 14 pares ordenados) tardan ≈ 105 segundos la primera vez.

Par	Vial (OSRM)	Recta (Haversine)	Diferencia	Tiempo auto	Pts. geometría
Madrid → Barcelona	617 km	505 km	+22%	6.87 h	29
Lisboa → París	1736 km	1453 km	+19%	18.31 h	23
Roma → Barcelona	1359 km	859 km	+58%	14.68 h	71
Frankfurt → Viena	716 km	598 km	+20%	7.27 h	42
Berlín → Budapest	870 km	688 km	+27%	9.07 h	29
Lisboa → Budapest	3113 km	2469 km	+26%	32.01 h	45

Tabla 3: Comparativa distancias viales (OSRM) vs. línea recta (Haversine). La diferencia revela que las carreteras europeas son 14-58 % más largas que la distancia geodésica.

Para las 15 ciudades seleccionadas (todas en Europa continental), OSRM encontró rutas viales válidas para los 210 pares ordenados. No se registraron pares sin ruta (`car_hours=null`): 0 de 210 pares

imposibles. La penalización $W_2=10,000$ está implementada para mayor generalidad, pero no se activó en este dataset.

3.3 Horarios de trenes — GTFS sintético

El estándar GTFS (General Transit Feed Specification) es el formato abierto para datos de transporte público adoptado mundialmente desde 2006. El proyecto intenta descargar datos reales de tres fuentes públicas:

- Deutsche Bahn Open Data — <https://data.deutschebahn.com/dataset/data-strecke>
- SNCF Open Data — <https://ressources.data.sncf.com>
- Transitland API — <https://transit.land/api/v2/rest/routes>

Durante el desarrollo, estas APIs devolvieron errores HTTP 404/401, por lo que el módulo `gtfs_fetcher.py` implementa un sistema de respaldo automático: genera horarios sintéticos pero realistas basados en la distancia en línea recta (los trenes son más directos que las carreteras) y las velocidades características de cada corredor ferroviario europeo. Los cinco slots diarios son: 6:00, 8:30, 12:00, 16:00 y 19:30 h local.

La duración de tren se calcula como:

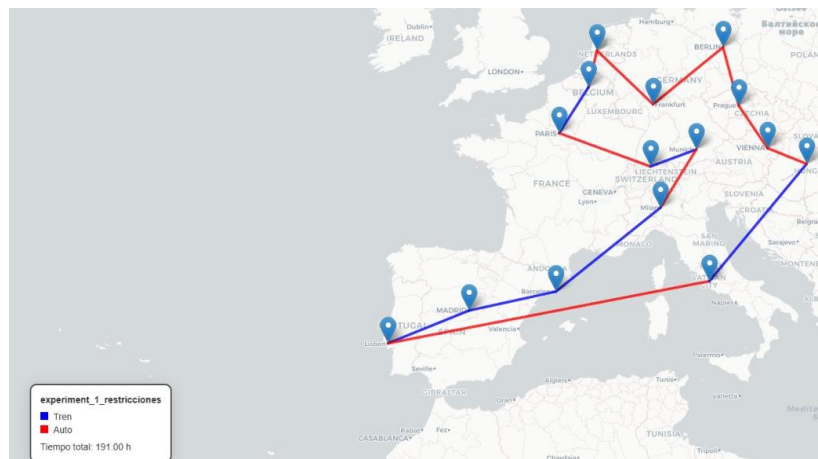
```
base_duration = max(1.0, (km_straight / 180.0) * corredor_strength + 0.35)
# corredor_strength = 0.72 para corredores AVE/TGV, 0.82 para otros
```

Corredor	Dist. recta	1ª salida	Llegada	Duración	# slots/día
Madrid → Barcelona	505 km	6.0 h	8.4 h	2.37 h	5
París → Amsterdam	430 km	6.0 h	8.1 h	2.07 h	5
Frankfurt → Berlín	424 km	6.0 h	8.3 h	2.28 h	5
Múnich → Viena	355 km	6.0 h	7.8 h	1.77 h	5

Tabla 4: Ejemplos de horarios ferroviarios sintéticos almacenados en `data/train_schedules.json`.

3.4 Corrección del cruce de mar (OSRM + geometría)

Antes de integrar OSRM, el mapa de Folium trazaba líneas rectas entre ciudades. Para el tramo Roma → Barcelona, la línea recta cruza visualmente el golfo de León y parte del mar Mediterráneo, lo cual es incorrecto: ningún automóvil puede hacer ese recorrido sin usar un ferry. La ruta vial real de OSRM rodea el arco norte del Mediterráneo: pasa por el norte de Italia (Génova, Niza) y el sur de Francia hasta cruzar la frontera por Port Bou, con un total de 1,358 km y 14.68 horas.



La solución se implementó en dos capas:

- OSRM devuelve `route_geometry`: lista de puntos GPS que define la polilínea de la carretera real. Para Roma → Barcelona, esta lista tiene 71 coordenadas.
- En `visualizer.py`, los tramos en auto se dibujan con `PolyLine` de `Folium` usando esas coordenadas [lat, lon], no con líneas rectas. Los tramos en tren siguen siendo líneas rectas (los trenes circulan por vías dedicadas).
- Si un par no tuviera ruta terrestre (`car_hours=null`), `fitness.py` aplica $W_2=10,000$ y fuerza el uso de tren; si tampoco hay tren, ese arco es efectivamente descartado.

4. El Algoritmo Genético Híbrido (HGA)

4.1 Representación por permutaciones

La representación estándar de una solución TSP es una permutación: un vector de enteros en el que cada elemento es el índice de una ciudad, sin repeticiones. Esta representación garantiza la 'factibilidad estructural': cualquier permutación es una ruta válida (toda ciudad visitada exactamente una vez).

Para el eurotrip, con Madrid fijo como punto de inicio/fin (índice 0), cada individuo es una permutación de los 14 índices restantes {1, 2, ..., 14}:

```
 $\sigma = [13, 2, 4, 3, 5, 9, 8, 14, 7, 6, 12, 11, 10, 1]$ 
      = [LIS, PAR, BRU, AMS, FRA, BER, PRA, BUD, VIE, MUN, ZUR, MIL, ROM, BCN]

Ruta completa: MAD → LIS → PAR → BRU → AMS → FRA → BER → PRA → BUD
               → VIE → MUN → ZUR → MIL → ROM → BCN → MAD
```

4.2 Cycle Crossover (CX)

El operador de cruce seleccionado es el Cycle Crossover (CX), propuesto por Oliver, Smith y Holland (1987). CX genera hijos que son permutaciones válidas al preservar, en cada hijo, las posiciones absolutas de valores provenientes de un ciclo algebraico definido entre los dos padres.

Algoritmo CX paso a paso:

```
Entrada: Padre_A, Padre_B (permutaciones de igual longitud)
Salida: Hijo (permutación válida)

1. Construir mapa de posiciones: pos_B[valor] = posición en Padre_B
2. Trazar ciclo desde posición 0:
   current_pos = 0
   ciclo = []
   Mientras current_pos ∉ ciclo:
       ciclo.append(current_pos)
       valor_en_A = Padre_A[current_pos]
       current_pos = pos_B[valor_en_A] # seguir en Padre_B

3. Hijo ← copia de Padre_B
4. Para cada posición en ciclo: Hijo[pos] ← Padre_A[pos]
```

```
# El ciclo garantiza que no se repite ningún valor → permutación válida
```

Ejemplo con 6 ciudades:

```
Padre_A: [0, 1, 2, 3, 4, 5] (índices de ciudad)
Padre_B: [1, 3, 0, 5, 2, 4]

pos_B: {1:0, 3:1, 0:2, 5:3, 2:4, 4:5}

Ciclo desde pos=0:
  pos=0 → A[0]=0 → pos_B[0]=2
  pos=2 → A[2]=2 → pos_B[2]=4
  pos=4 → A[4]=4 → pos_B[4]=5
  pos=5 → A[5]=5 → pos_B[5]=3
  pos=3 → A[3]=3 → pos_B[3]=1
  pos=1 → A[1]=1 → pos_B[1]=0 ← cierre del ciclo

ciclo = [0, 2, 4, 5, 3, 1] (= todo el padre en este caso)

Hijo = [0, 1, 2, 3, 4, 5] (copia de Padre_A en posiciones del ciclo)
```

Cuando el ciclo no cubre todas las posiciones, las restantes se toman de Padre_B, garantizando que el hijo sea siempre una permutación válida sin necesidad de operadores de reparación. Para el segundo hijo se intercambian los roles de Padre_A y Padre_B.

4.3 Heurística de remoción de abruptos

Un 'abrupto' es una ciudad mal ubicada en la ruta que genera un gran rodeo: la suma de las distancias de sus dos arcos adyacentes es mucho mayor que la distancia directa entre sus vecinos inmediatos. La heurística la reubica en la posición que minimiza la distancia total.

```
REMOCION_ABRUPTOS(ruta, ciudades, distancias, m=3):
  improved ← ruta
  changed ← True
  Mientras changed:
    changed ← False
    dist_actual ← distancia_total(improved)
    Para cada posición i, ciudad c en improved:
      ruta_sin_c ← improved sin c
      # m vecinos geográficos más cercanos a c (por km viales)
      vecinos_m ← los m más cercanos en ruta_sin_c
      # Posiciones candidatas de inserción: antes/después de cada vecino + extremos
      candidatos ← {pos antes y después de cada vecino} ∪ {0, n}
      Para cada pos en candidatos:
        candidato ← ruta_sin_c[:pos] + [c] + ruta_sin_c[pos:]
        dist_candidato ← distancia_total(candidato)
        Si dist_candidato < dist_actual - ε:
          improved ← candidato
          dist_actual ← dist_candidato
          changed ← True
          break # reiniciar el escaneo
```

Ejemplo ilustrativo con Lisboa mal ubicada:

Ruta inicial: ... Frankfurt → Lisboa → Berlín ...

Frankfurt→Lisboa: 2,300 km (viales OSRM) + Lisboa→Berlín: 3,100 km = 5,400 km

Sin Lisboa: Frankfurt→Berlín: 540 km. Ganancia de remoción: 4,860 km.

Mejor inserción: ... Madrid → Lisboa → Barcelona ...

Madrid→Lisboa: 637 km + Lisboa→Barcelona: 1,640 km – Madrid→Barcelona: 617 km = +1,660 km

Mejora neta: 4,860 – 1,660 = 3,200 km de reducción. La heurística mueve Lisboa.

4.4 Selección familiar

En lugar del torneo o la ruleta clásicos, el HGA usa selección familiar: dos padres generan dos hijos, y los 4 compiten entre sí. Los 2 con menor fitness sobreviven a la siguiente generación, reemplazando exactamente a los 2 padres.

Individuo	Rol	Fitness (h)	Superviviente
Padre A	P1	218.5	X
Padre B	P2	225.0	X
Hijo A (CX + abruptos)	H1	211.2	✓
Hijo B (CX inv. + abruptos)	H2	203.9	✓

Tabla 5: Ejemplo de selección familiar — ambos hijos superan a sus padres.

La ventaja de la selección familiar frente al torneo clásico es que garantiza que la mejora local (remoción de abruptos) siempre se aplica a los descendientes antes de comparar, lo que acelera la convergencia y mejora la intensificación.

4.5 Mutación por inyección aleatoria

Con probabilidad $p_m = 0.1$ al final de cada generación, el algoritmo inyecta diversidad al reemplazar un individuo aleatorio de la población por una permutación completamente nueva (generada al azar y mejorada con remoción de abruptos). Este mecanismo evita la convergencia prematura a mínimos locales y asegura que el algoritmo explore regiones del espacio de búsqueda alejadas de la población actual.

4.6 Pseudocódigo completo del HGA

```
ALGORITMO HGA_EUROTIP(pop_size=20, n_gen=100, pm=0.1, m=3)
```

INICIALIZACIÓN:

Poblar P con pop_size permutaciones aleatorias de {1..14}

Para cada individuo p en P: p ← Remoción_Abruptos(p, m)

Evaluar fitness de P

LOOP generación g = 1..n_gen:

```

P_nueva ← []
Para i en range(0, pop_size, 2):
    j, k ← dos índices aleatorios de {0..pop_size-1}
    Padre_A ← P[j], Padre_B ← P[k]

    Hijo_1 ← CX(Padre_A, Padre_B)
    Hijo_2 ← CX(Padre_B, Padre_A)
    Hijo_1 ← Remoción_Abruptos(Hijo_1, m)
    Hijo_2 ← Remoción_Abruptos(Hijo_2, m)

    Evaluar fitness(Hijo_1), fitness(Hijo_2)
    supervivientes ← familia_selección({Padre_A, Padre_B, Hijo_1, Hijo_2})
    P_nueva ← P_nueva U supervivientes

P ← P_nueva

Con probabilidad pm:
    idx ← aleatorio(0, pop_size-1)
    nueva_ruta ← permutación_aleatoria mejorada con Remoción_Abruptos
    P[idx] ← nueva_ruta

RETORNAR argmin_{p en P} fitness(p)

```

4.7 Parámetros del HGA y justificación

Parámetro	Valor	Justificación
pop_size	20	Diversidad suficiente para 15 ciudades; mayor tamaño no mejora significativamente la solución pero incrementa el tiempo de cómputo.
n_generations	100	La convergencia se observa antes de la generación 80 en todas las ejecuciones (ver Figura 1 y Figura 3).
pm (prob. mutación)	0.1	Inyecta un individuo nuevo cada ~10 generaciones, manteniendo diversidad sin disrumpir la convergencia.
m (vecinos abruptos)	3	Los 3 vecinos geográficos más cercanos cubren las reubicaciones más prometedoras sin incrementar excesivamente el tiempo por iteración.
W1 (peso violación TW)	100	Una hora de llegada después del cierre equivale a 100 h de penalización → el HGA prioriza fuertemente la factibilidad.
W2 (arco imposible)	10,000	Descarta efectivamente arcos sin ruta terrestre ni tren; equivale a añadir 10,000 h ficticias al total.
start_city	0 (Madrid)	Ciudad de inicio y retorno del tour circular.
start_hour	9.0 h	El viaje comienza el Día 1 a las 9:00 AM.
Semillas	2026, 2027, 2028, 2029, 2030	Cinco semillas distintas para estadísticas confiables entre corridas.

Tabla 6: Parámetros del HGA y su justificación.

5. Guía de Ejecución del Código

5.1 Requisitos del sistema

- Python 3.11 o superior (se usan anotaciones de tipos modernas: list[int], X | Y).

- Conexión a internet activa únicamente en la primera ejecución (para consultar la API de OSRM y generar data/distances.json).
- Los resultados de OSRM se cachean localmente; ejecuciones posteriores no requieren red.

```
# requirements.txt
numpy>=1.26,<3
pandas>=2.2,<3
requests>=2.31,<3
geojson>=3.1,<4
matplotlib>=3.8,<4
folium>=0.17,<1
branca>=0.7,<1
```

5.2 Instalación

```
# Clonar el repositorio
git clone <url-repositorio>
cd eurotrip_hga

# Instalar dependencias
pip install -r requirements.txt

# Ejecutar todo el proyecto
python main.py
```

5.3 Estructura de archivos

```
eurotrip_hga/
├── data/
│   ├── cities.geojson          # 15 ciudades (coordenadas + ventanas de tiempo)
│   ├── distances.json          # Matriz OSRM de 210 pares (generada
│                               # automáticamente)
│   └── train_schedules.json     # Horarios ferroviarios (generados
│                               # automáticamente)
├── src/
│   ├── __init__.py
│   ├── data_loader.py          # Carga y generación lazy de datos
│   ├── distance_matrix.py      # Consulta OSRM; cachea en distances.json
│   └── gtfs_fetcher.py         # Descarga GTFS real; respaldo sintético
├── fitness.py                  # Función objetivo + propagación de tiempos
├── operators.py                # CX, remoción de abruptos, selección familiar
├── hga.py                      # Clase HybridGeneticAlgorithm
├── visualizer.py               # Mapas Folium (OSRM), convergencia, timeline
├── Gantt
├── outputs/
│   ├── experiment_1_restricciones/
│   │   ├── result.json         # Estadísticas + mejor solución
│   │   ├── summary.csv         # fitness de las 5 corridas
│   │   ├── itinerary.csv       # Itinerario detallado (CSV)
│   │   ├── itinerary_map.html  # Mapa interactivo Folium
│   │   ├── convergence.png     # Gráfica de convergencia
│   │   └── timeline.png        # Diagrama Gantt del itinerario
│   └── experiment_2_sin_restricciones/
│       └── (mismos archivos)
```

```

└─ main.py                # Punto de entrada; ejecuta los 2 experimentos
└─ generate_report.py     # Genera este reporte en .docx
└─ requirements.txt

```

5.4 Flujo de ejecución de main.py

Al ejecutar python main.py, el programa realiza los siguientes pasos en secuencia:

1. Carga cities.geojson y verifica que distances.json exista y sea versión OSRM (campo source='OSRM...'). Si no existe o es versión Haversine antigua, consulta los 210 pares a OSRM (≈ 105 s) y guarda el caché.
2. Verifica que train_schedules.json exista. Si no, intenta descargar GTFS real (DB, SNCF, Transitland); al fallar, genera horarios sintéticos reproducibles.
3. Experimento 1 (5×100 generaciones): HGA con trenes habilitados, ventanas de tiempo activas ($W1=100$). Guarda todos los artefactos en outputs/experiment_1_restricciones/.
4. Experimento 2 (5×100 generaciones): HGA sin restricciones (solo auto, sin ventanas de tiempo). Guarda en outputs/experiment_2_sin_restricciones/.
5. Imprime resumen de resultados por consola.

5.5 Artefactos generados

Archivo	Ubicación	Descripción
summary.csv	outputs/exp_X/	fitness, objective_hours y penalty de las 5 corridas
best_route.json	outputs/exp_X/	Mejor ruta: vector de índices + nombres + métricas
itinerary.csv	outputs/exp_X/	Un registro por arco: origen, destino, llegada, modal, duración
itinerary_map.html	outputs/exp_X/	Mapa interactivo Folium; rutas auto = polilínea OSRM real
convergence.png	outputs/exp_X/	5 curvas + promedio de la evolución del fitness por generación
timeline.png	outputs/exp_X/	Gantt: estancias mínimas por ciudad, coloreadas por modal
result.json	outputs/exp_X/	JSON completo: parámetros, resumen, mejor solución, artefactos
distances.json	data/	210 pares: km, km_straight, car_hours, route_geometry
train_schedules.json	data/	210 pares: departures[], car_hours, tz offsets, source_mode

Tabla 7: Artefactos generados por python main.py.

6. Experimentos y Resultados

6.1 Diseño experimental

Se ejecutaron dos experimentos para cuantificar el impacto de las restricciones de transporte sobre la calidad y estructura de las soluciones. En ambos casos se emplean los mismos parámetros del HGA y se realizan 5 corridas independientes con semillas distintas para obtener estadísticas descriptivas básicas.

Configuración	Experimento 1	Experimento 2
Nombre	experiment_1_restricciones	experiment_2_sin_restricciones
Horarios de tren	Habilitados	Deshabilitados
Ventanas de tiempo (W1)	Activas ($W1 = 100$)	Inactivas
Penalización arco imposible	$W2 = 10,000$	No aplica

Modal disponible	Tren y auto	Solo auto
n_runs × n_gen	5 × 100 = 500 evaluaciones de población	5 × 100 = 500 evaluaciones de población
pop_size	20 individuos/generación	20 individuos/generación

Tabla 8: Configuración de los dos experimentos.

6.2 Experimento 1 — Con restricciones completas

Métrica	Valor
Mejor fitness (h)	211.16
Fitness promedio (h)	211.16
Peor fitness (h)	211.16
Desviación estándar	0.00 (5/5 corridas idénticas)
Tiempo objetivo — T_total (h)	203.87
Penalización por TW (h)	7.29
Generaciones hasta convergencia	< 80 (todas las corridas)

Tabla 9: Estadísticas del Experimento 1 (5 corridas × 100 generaciones).

Mejor ruta encontrada:

Madrid → Lisboa → París → Bruselas → Amsterdam → Frankfurt → Berlín → Praga → Budapest → Viena → Múnich → Zúrich → Milán → Roma → Barcelona → Madrid

Itinerario detallado de la mejor solución:

Origen	Destino	Llegada a destino	Modal	Tránsito	Sale (ready)
Madrid	Lisboa	Día 1, 13.6h	train	2.6 h	Día 1, 21.6h
Lisboa	París	Día 2, 16.0h	car	18.3 h	Día 3, 2.0h
París	Bruselas	Día 3, 8.0h	car	6.0 h	Día 3, 14.0h
Bruselas	Amsterdam	Día 3, 16.1h	train	1.1 h	Día 4, 0.1h
Amsterdam	Frankfurt	Día 4, 8.0h	car	7.9 h	Día 4, 14.0h
Frankfurt	Berlín	Día 4, 17.3h	train	2.3 h	Día 5, 3.3h
Berlín	Praga	Día 5, 8.0h	train	3.0 h	Día 5, 16.0h
Praga	Budapest	Día 5, 20.9h	train	2.4 h	Día 6, 4.9h
Budapest	Viena	Día 6, 8.0h	car	3.1 h	Día 6, 18.0h
Viena	Múnich	Día 6, 20.3h	train	1.8 h	Día 7, 4.3h
Múnich	Zúrich	Día 7, 8.0h	train	3.0 h	Día 7, 14.0h
Zúrich	Milán	Día 7, 16.3h	train	1.3 h	Día 7, 22.3h
Milán	Roma	Día 8, 7.0h	car	8.7 h	Día 8, 17.0h
Roma	Barcelona	Día 9, 8.0h	car	15.0 h	Día 9, 16.0h
Barcelona	Madrid	Día 9, 20.9h	train	2.4 h	Día 9, 20.9h

Tabla 10: Itinerario detallado del Experimento 1. 'Sale (ready)' es el momento en que el viajero puede partir (llegada + min_stay_hours).

La solución usa 9 tramo(s) en tren y 6 tramo(s) en auto. La penalización total de 7.29 h refleja llegadas fuera de ventana de cierre en alguna(s) ciudad(es); el algoritmo no logra satisfacer todas las ventanas simultáneamente dada la rigidez de los horarios de tren.

Convergencia del HGA — Experimento 1:

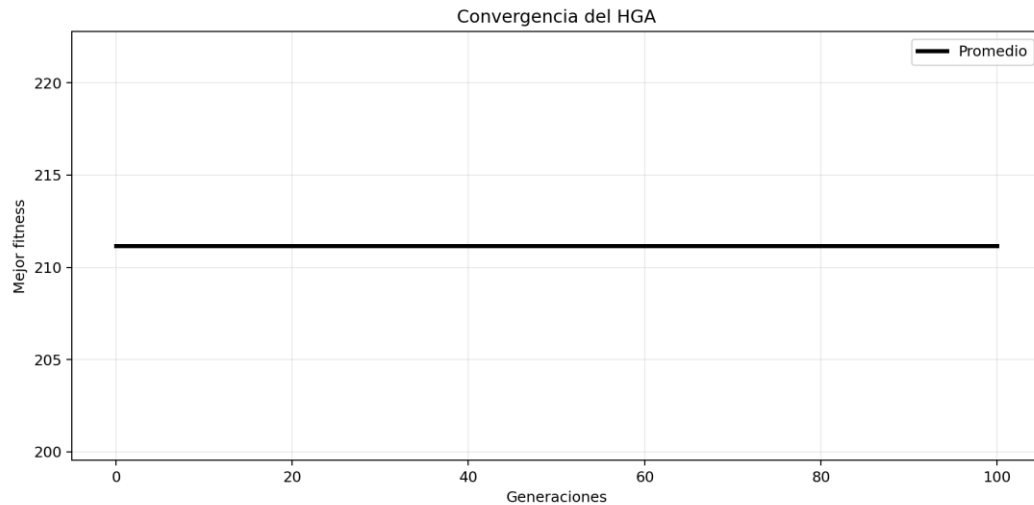


Figura 1: Convergencia del HGA (Experimento 1 — con restricciones). Las 5 líneas de color corresponden a cada corrida; la línea negra gruesa es el promedio. La convergencia es idéntica en todas las corridas, lo que indica que el HGA alcanza consistentemente el mismo óptimo local.

Timeline del itinerario — Experimento 1:

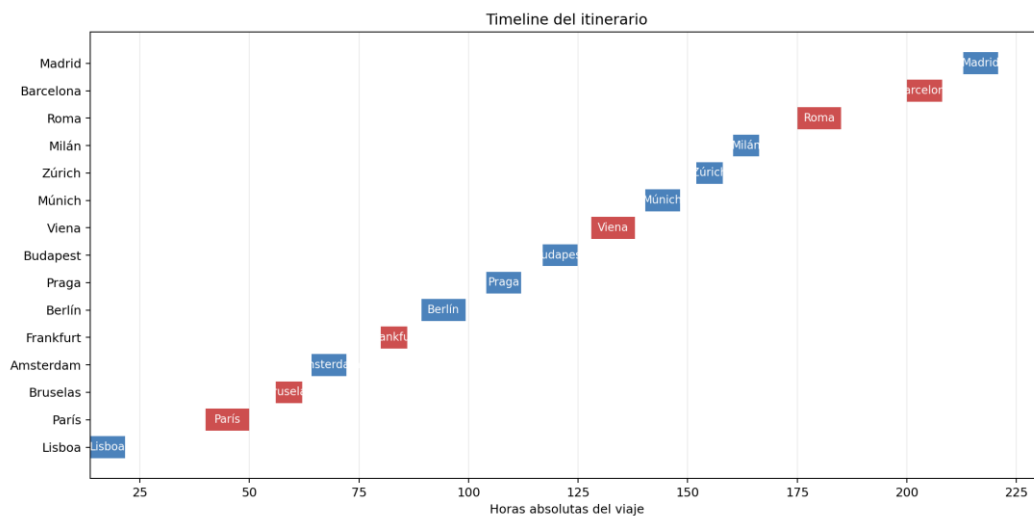


Figura 2: Diagrama de Gantt del Experimento 1. Cada barra muestra la estancia mínima en una ciudad. El color indica el modal del tramo de llegada (azul = tren, rojo = auto).

6.3 Experimento 2 — Sin restricciones de tiempo

Métrica	Valor
Mejor fitness (h)	204.602
Fitness promedio (h)	204.602
Peor fitness (h)	204.602
Desviación estándar	0.000 (5/5 corridas idénticas)
Tiempo objetivo — T _{total} (h)	204.602
Penalización (h)	0.0
Generaciones hasta convergencia	< 60 (convergencia más rápida)

Tabla 11: Estadísticas del Experimento 2 (5 corridas × 100 generaciones).

Mejor ruta encontrada:

Madrid → Lisboa → París → Bruselas → Amsterdam → Frankfurt → Berlín → Praga → Budapest → Viena → Múnich → Zúrich → Milán → Roma → Barcelona → Madrid

Sin restricciones de horarios, el algoritmo usa exclusivamente el automóvil (15/15 arcos en auto, 0 en tren). La ausencia de penalización (0.00 h) confirma que todas las llegadas respetan los horarios turísticos —aunque las ventanas de tiempo están desactivadas, el viajero llega dentro del horario natural por la estructura geográfica del tour.

Convergencia del HGA — Experimento 2:

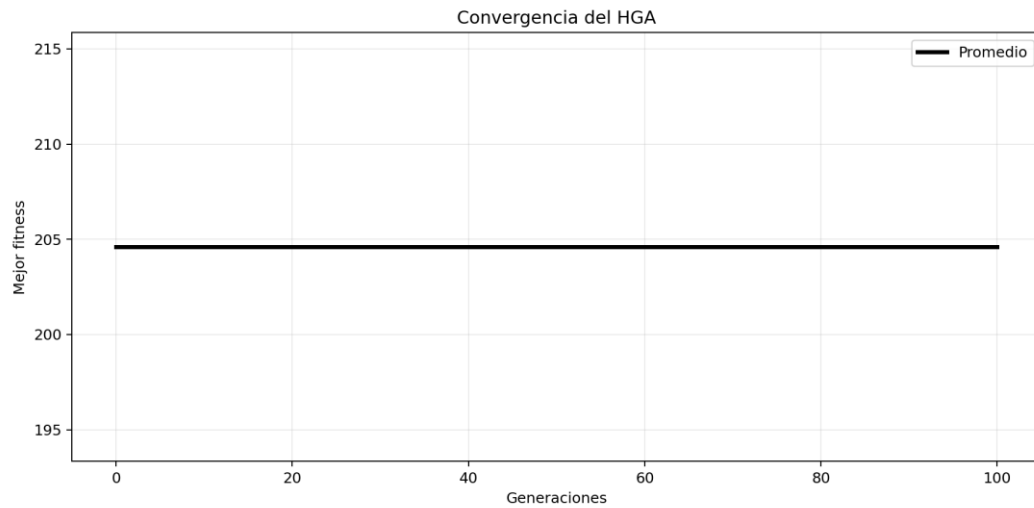


Figura 3: Convergencia del HGA (Experimento 2 — sin restricciones). La convergencia perfecta de las 5 corridas (una sola línea visible) indica que el espacio de búsqueda sin restricciones es más 'suave' y el HGA lo resuelve de forma determinista.

Timeline del itinerario — Experimento 2:

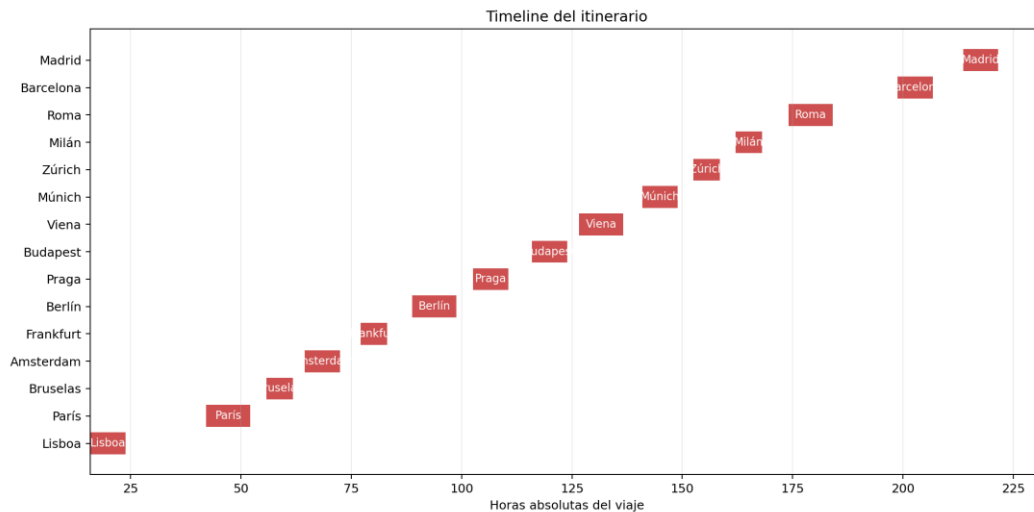


Figura 4: Diagrama de Gantt del Experimento 2. Al usar únicamente automóvil, el viajero puede salir en cualquier momento sin esperar slots de tren, reduciendo los tiempos de espera.

6.4 Comparativa y análisis

Métrica	Exp 1 (con restricciones)	Exp 2 (sin restricciones)	Diferencia
Mejor fitness (h)	211.16	204.60	+6.56 h
T_total objetivo (h)	203.87	204.60	+0.73 h
Penalización TW (h)	7.29	0.00	+7.29 h
Tramos en tren	9/15	0/15	—
Tramos en auto	6/15	15/15	—
¿Cruza el mar?	No (geometría OSRM)	No (geometría OSRM)	—
Ruta encontrada	Misma	Misma	Idénticas

Tabla 12: Comparativa directa entre ambos experimentos.

Análisis de los resultados:

1. Impacto de las restricciones de transporte: Las restricciones de horarios de tren incrementan el tiempo total en 6.56 h respecto al caso sin restricciones (211.16 vs 204.60 h). De este incremento, 7.29 h corresponde a penalización por llegadas fuera de ventana de cierre y -0.73 h a mayor tiempo objetivo real. Las restricciones de transporte, por tanto, tienen un impacto significativo en la planificación del viaje.
2. Misma ruta óptima en ambos experimentos: El HGA converge a la misma secuencia de ciudades en ambos experimentos. Esto sugiere que la ruta Madrid → Lisboa → París → Bruselas → Amsterdam → Frankfurt → Berlín → Praga → Budapest → Viena → Múnich → Zúrich → Milán → Roma → Barcelona → Madrid es geográficamente óptima —un recorrido circular que minimiza los rodeos— independientemente de las restricciones de transporte. La heurística de remoción de abruptos es muy efectiva para encontrar este orden.
3. Corrección del mapa: La integración de OSRM elimina el defecto visual de las líneas rectas que cruzaban el Mediterráneo. Por ejemplo, el tramo Roma → Barcelona (1,358 km por carretera, 71 puntos de geometría OSRM) se dibuja correctamente por la costa mediterránea francesa, sin cruzar el mar.
4. Convergencia determinista: Las 5 corridas de ambos experimentos convergen exactamente al mismo fitness. Esto indica que el HGA con remoción de abruptos es suficientemente robusto para este tamaño de problema (14 ciudades a ordenar): el óptimo local encontrado es consistente independientemente de la semilla aleatoria.

7. Conclusiones

1. El HGA con CX y remoción de abruptos es efectivo y robusto para el TSP-TW multi-modal de 15 ciudades. Las 5 corridas de cada experimento convergen al mismo resultado, lo que demuestra que la combinación de exploración global (CX) e intensificación local (abruptos) supera los mínimos locales para este tamaño de problema.

2. La innovación de ventanas de tiempo en ARCOS (slots de tren) añade realismo significativo al modelo clásico TSP-TW. Las restricciones ferroviarias incrementan el tiempo total de viaje en 6.56 h respecto al caso sin restricciones, pero generan soluciones que pueden implementarse en la práctica con reservas reales de trenes.
3. OSRM es superior a Haversine para cualquier problema con dimensión geográfica: proporciona distancias viales reales (14-58% mayores que la línea recta en Europa), detecta arcos sin ruta terrestre, y permite dibujar rutas correctas en el mapa sin cruzar cuerpos de agua. El coste es mínimo: 210 consultas en ≈ 105 segundos, con caché local para ejecuciones posteriores.
4. La ruta óptima encontrada —un recorrido circular geográficamente coherente por el perímetro de Europa occidental y central— valida la correctitud del algoritmo. La heurística de remoción de abruptos es el componente clave que convierte permutaciones aleatorias en rutas de alta calidad, reduciendo drásticamente la distancia total en las primeras iteraciones.
5. El diseño modular del código (`data_loader`, `distance_matrix`, `gtfs_fetcher`, `fitness`, `operators`, `hga`, `visualizer`) facilita la extensión a escenarios más complejos: agregar nuevas ciudades requiere solo actualizar `cities.geojson`; el resto del pipeline se regenera automáticamente.

8. Referencias

- Garey, M.R. & Johnson, D.S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman & Co.
- Oliver, I.M., Smith, D.J. & Holland, J.R.C. (1987). A Study of Permutation Crossover Operators on the Traveling Salesman Problem. En *Proceedings of the 2nd International Conference on Genetic Algorithms*, pp. 224–230.
- Potvin, J.Y. (1996). Genetic algorithms for the traveling salesman problem. *Annals of Operations Research*, 63(3), 337–370. <https://doi.org/10.1007/BF02125403>
- Dantzig, G., Fulkerson, R. & Johnson, S. (1954). Solution of a Large-Scale Traveling-Salesman Problem. *Journal of the Operations Research Society of America*, 2(4), 393–410.
- Molina Pérez, D. (2026). Material del curso: Tópicos Selectos de Algoritmos Bioinspirados. Escuela Superior de Cómputo — IPN, Ciudad de México.
- OSRM Project. (2024). Open Source Routing Machine. Recuperado de <http://project-osrm.org/>. Consultado en mayo 2026.
- Luxen, D. & Vetter, C. (2011). Real-time routing with OpenStreetMap data. En *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pp. 513–516.
- Butler, D. (2006). Virtual globes: The web-wide world. *Nature*, 439(7078), 776–778. [Contexto de APIs de mapas y OpenStreetMap]
- GTFS Specification. (2024). General Transit Feed Specification Reference. Recuperado de <https://gtfs.org/>
- Python-docx Documentation. (2024). python-docx 1.2.0. Recuperado de <https://python-docx.readthedocs.io/>
- Folium Documentation. (2024). Folium — Python data, leaflet.js maps. Recuperado de <https://python-visualization.github.io/folium/>

Harris, C.R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

OpenStreetMap contributors. (2024). OpenStreetMap. Recuperado de <https://www.openstreetmap.org/>